Build and Ship a Full-Stack App with AI Coding Assistants

Part 2 of the AI Dev Tools Zoomcamp series



ALEXEY GRIGOREV
AUG 03, 2026

This is the second article in a series based on AI Dev Tools Zoomcamp, the free course we run at DataTalks.Club.

In the first article, I wrote about turning an idea into a specification. The specification is only the beginning. In this article, we will create an end-to-end application from scratch. We will cover both frontend and backend, and add a database.

All articles in the series:

- Part 1: AI-Native Development: Specifications, Loop and Graph Engineering
- Part 2: Build and Ship a Full-Stack App with AI Coding Assistants (this article)
- Part 3: Deploy a Full-Stack App with AI Coding Assistants
- Part 4: DevOps and Observability for an AI-Built App
- Part 5: Coding Agent Building Blocks: Reusable Skills and Specialized Subagents

Thanks for reading Alexey On Data!
Subscribe for free to receive new posts and
support my work.

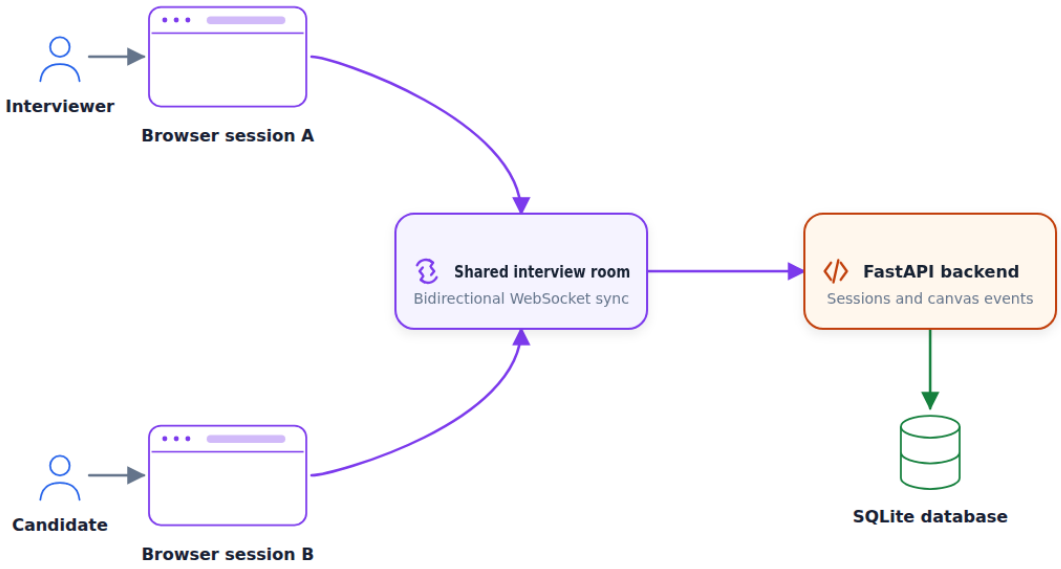
Subscribed



We will create an application for system-design interviews.

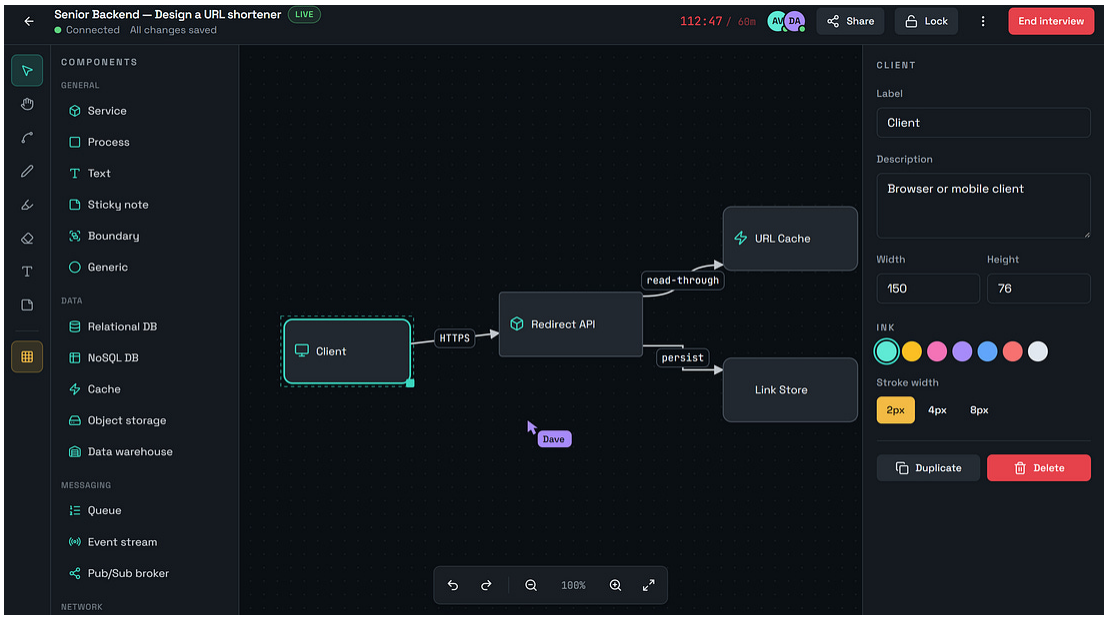
As an interviewer, you create a session and share the link with the interviewee. They can create diagrams in the app, and you see the changes in real time.

Both frontends connect to the same room through WebSockets, so the changes appear in both sessions simultaneously. The FastAPI backend handles those events and saves them to SQLite.



Two browser sessions collaborate through one WebSocket-backed interview room

You can find the code in the interview-canvas-share repository.



AI System Design Canvas: the application we will develop.

It's based on my full-day workshop I did for AI Shipping Labs: Build and Deploy a Full-Stack App with AI Coding Assistants.

Overview

Like in the previous article, we start with an idea. As this is only an idea, we turn it into a specification using ChatGPT.

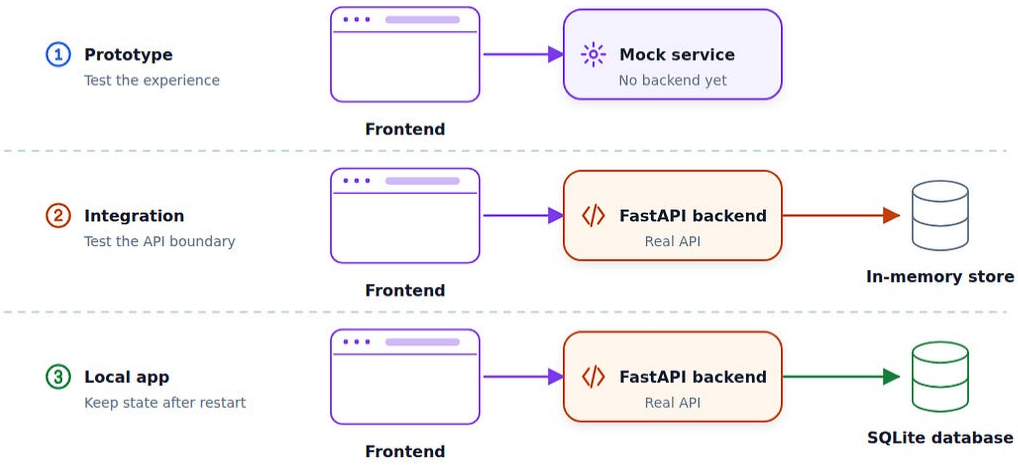
From there, we start building:

1. Generate a frontend that uses mocked backend calls.
2. Establish backend-frontend contract by creating OpenAPI specifications from the frontend service layer.
3. Implement the FastAPI backend, connect it to the frontend, and test the application.
4. Add persistence with SQLite and SQLAlchemy.

At each stage we get something concrete that we can test:

- First we define the specification and make sure it reflects what we want to build.
- After that, we use the specs to create a frontend prototype that we can interact with. We mock backend calls so we can test the idea.
- Then we connect the frontend to the backend. We start with an in-memory store to make sure the frontend-backend connection works.
- Finally, we add a database so the application keeps its state after a restart.

At the end, we have a working local application that is ready for deployment.



The interfaces stay stable while temporary components are replaced one at a time

Start with a specification

Before building the frontend, we need to describe the application precisely. If we don't do it, we will get something that works but we don't need.

For our application, we need to specify:

- who creates an interview session
- how a candidate joins it
- which components they can place on the canvas
- how both people see changes in real time

This is called "Specification-Driven Development". We don't spend a lot of time here because it was the focus of the previous article AI-Native Development: Specifications, Loop and Graph Engineering.

For creating the specification, I always use ChatGPT in dictation mode. Give the assistant as much information as possible at this stage.



Dictating the initial application idea to ChatGPT

You can see the result here.

Frontend First

Now we have the specification, but it's only text. There are multiple options of what you can do next:

- Focus on the database layer, define the entities, and work all the way up through backend to frontend
- Alternatively, you can start with specifying OpenAPI and define how backend and frontend interact and build both independently from there
- Or you can focus on the frontend first and then build the rest

All these approaches make sense and have their pros and cons.

For simple projects that you're building alone, I recommend starting with frontend. You can quickly judge if you're moving in the right direction, and if it solves your problem or not.

Treat it as a way to test your idea and your specification, and do it before you build the rest of the application.

For this step I like using Lovable, which generates a React app from one prompt. Claude Design, Replit, v0, and Bolt are similar tools that you can use instead.

Alternatively, you can start with coding assistants directly and ask them to create a React app (or whatever technology you want).

I use Lovable because it creates really nice designs. Also, I'm not a frontend engineer and I don't know much about the frontend world. Lovable makes technology choices for me that I know will make sense, while a coding agent can select something randomly.

Now open any tool of your choice and give it this prompt:

```
Create a system design interview application.

[Paste the ChatGPT-generated specification here.]

Centralize every backend call in one services layer, and create a mock
implementation of it so the whole app runs without a real backend.

Add tests.
```

This part is quite important:

> Centralize every backend call in one services layer, and create a mock implementation of it so the whole app runs without a real backend.

Without it, the agent can do something arbitrary, but here we explicitly say that we want a mock service. Later, it will become the single point of integration of our frontend with backend. And because we mock it, it will work from the beginning.



The frontend is real and interactive; only its backend service is mocked

Lovable creates a React app in TypeScript. We can interact with it in the Lovable interface.

Next, save it to GitHub. If you use Lovable:

- Select the plus icon in the bottom-left corner
- Connect your GitHub account.
- Lovable creates a private repository. I usually change it to public.

Move the Frontend into the Project

Next, we can clone the repository locally.

For the rest of the project, I want this setup:

```
/backend    # backend application and its tests
/docs       # supporting documentation
/frontend   # frontend application
AGENTS.md   # instructions for coding agents
openapi.yaml # API agreement
```

So let's create these folders and move all the frontend stuff to "frontend", and the specification we created to "/docs/spec.md".

After we re-arranged the files, commit the changes.

At this point, you should also be able to run the application locally and test that things work the way you want. If they don't, use a coding agent to fix it.

To run a project you exported from Lovable:

```
cd frontend
npm i
npm run dev
```

AGENTS.md

We already discussed the importance of `AGENTS.md` in the first article.

Let's create one for this project too. Place it in the repository root:

```
for backend, use uv for dependency management. a few useful commands:

uv sync
uv add <PACKAGE-NAME>
uv run python <PYTHON-FILE>

regularly commit code to git
```

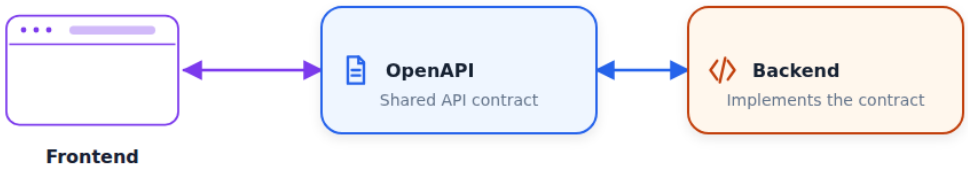
This is only the starting point and it will change as your project grows.

OpenAPI Specifications

When creating frontend, we asked the AI assistant to put everything in a centralized service layer. Later we will replace it with actual calls to backend.

But now we should define the specification - the agreement between frontend and backend. We will use OpenAPI for that.

This specification gives explicit information about the endpoints, paths, request bodies, response bodies, and authentication rules.



OpenAPI is the explicit contract shared by the frontend and backend

Let's create it:

> Read the frontend's API client in frontend/
>
> Create openapi.yaml at the repository root.
>
> Specify the backend this frontend expects: every endpoint, method, path, request body, response body, and which endpoints need authentication.

You can skip this step. But I wouldn't recommend it.

It takes a few minutes, but has many benefits. The backend gets a precise target instead of being inferred from frontend code. Not only we save tokens this way, but also get a clear picture of what exactly the backend needs.

You can see the result in openapi.yaml.

The Backend

Now we have the OpenAPI specs and we can use this file to create the backend. We will use Python and FastAPI for that. But you can choose any technology you want.

When I start with frontend, I use a mocked backend and then replace it with a real one. In the same way, for FastAPI backends, I start with a mocked database, and then later change it.

I do that because I want to make sure the frontend-backend connection works, and until it's smooth, I don't worry about the persistence.

Let's ask the coding assistant to implement it:

```
Build a FastAPI backend in backend/ that implements the openapi.yaml spec.

Use an in-memory store and seed it with data so the frontend has something to
show. Add authentication with hashed passwords and bearer tokens for the
endpoints that need it.

Split the code into modules - routers, models, store, auth

Write tests
```

After 5-10 minutes, you will have the backend ready.

Makefile

When it's ready, you can run the application. Normally, for FastAPI, the command is something like that:

```
cd backend
uv run uvicorn backend.main:app --reload --port 8091
```

I'll use port 8091 but you can replace it with any other port you like.

But for me it's always hard to remember these commands, so I ask the coding assistant to create a Makefile:

> Create a Makefile so I can easily run it.

Then running it is as simple as

```
make run
```

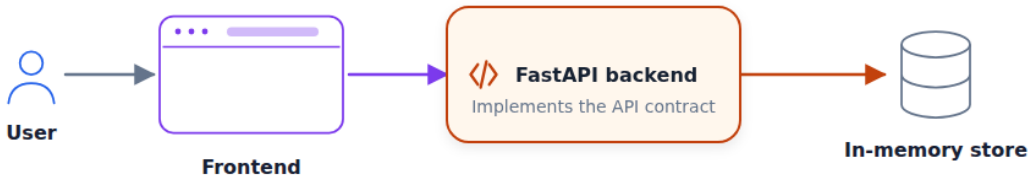
When it's running, open http://localhost:8091/docs in the browser. You will see the OpenAPI specification of the implemented backend.

We can compare it with the actual specs. From this point we no longer need the original `openapi.yaml` - the one that's generated by FastAPI is enough.

Connecting Frontend and Backend

The backend is runnable now, so we can connect it to the frontend:

> Switch the frontend to use the real backend client.



Connect the real frontend and backend while keeping persistence mocked

It most likely won't work from the first try. You will hist CORS errors and probably some others.

It's also time to test the application thoroughly. Open two browser windows:

- Create a session in one and get the join link
- Use the join link in the second window
- Interact with the system in the second window and see the changes being propagated to the first one.

If something doesn't work, ask the agent to fix it.

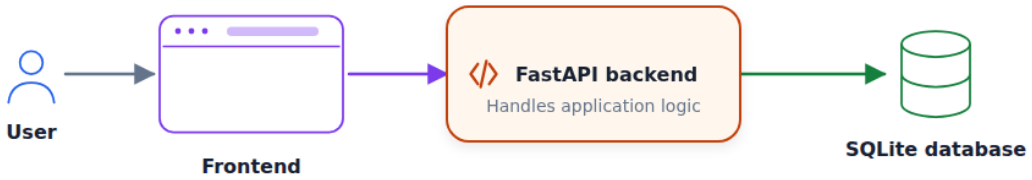
Database

At this point we have a working application with frontend and backend. They can connect to each other.

But because we don't have a real database yet, when the backend is restarted, all the data is lost.

Let's fix it:

> Replace the in-memory store with a database. Use SQLite and SQLAlchemy.
> Use an environment variable to configure which DB the server should connect to.
> Make it database-agnostic - later we will add support for other databases (e.g. Postgres).



SQLite replaces the in-memory store without changing the rest of the application flow

There are a few important parts in this prompt:

- First, I ask it to use SQLite. It's a very nice and lightweight database for testing the application locally.
- At some point, you'll want to switch to a different database for production, e.g. Postgres. Using SQLAlchemy makes this process very simple.
- But we also mention that explicitly so the agent doesn't use any SQLite specific features in the first version.

After 5-10 minutes you will have a fully working end-to-end application that's running locally. Now you can test it: create multiple sessions and see that changes in one are reflected in another. Stop your backend server, start again, and check that all the changes are still there.

If something doesn't work, ask the coding assistant to fix it.

Ready for Deployment

Now we have a working application:

- we turned an idea into a specification
- created frontend

- defined the API contract
- implemented the backend from the contract
- connected them
- added SQLite for persistence

But this application is only working locally. Next, we need to deploy it. That's something we will do in the next article. We will take what we developed here and turn it into a deployed service.

We will:

- containerize the frontend and backend
- add integration tests for the full application flow
- set up CI to run the checks automatically
- run Postgres with Docker Compose
- add database migrations
- deploy the application to a public environment
- set up CD to deploy changes after the checks pass

Thanks for reading Alexey On Data!
Subscribe for free to receive new posts and
support my work.

Subscribed

Step-by-Step Approach

In this workshop, we created a working application using the step-by-step approach. At the end of each step, we had something functioning:

- Clear speciation

- Working frontend prototype with mock backend
- Properly integrated backend with mock database
- Persistence with SQLite

For each step, we started from a new session and drove the agent to produce a working version.

But we did it one prompt at a time. After the first version is working, it's time to introduce the development process. We talked about it in the first article. This process will help you application continue working properly as it grows bigger.

Next in the Series

In the next modules we continue working with this app:

- Deployment: deploy the application to the cloud and set up CI/CD
- DevOps and Observability: add observability to the application and use AI as the first line of support when the application breaks

You can find the entire course in the course repository. It's free.

⭐ Star the course repo

This article is based on the the workshop I did for AI Shipping Labs: Build and Deploy a Full-Stack App with AI Coding Assistants. In that workshop I created the snake game, deployed it with AWS and configured CI/CD to make sure tests run on every push. Check it out!